

Homework 02

Question 1: Training and test error under a fixed design

Answer

Part a

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(43202)

n, p = 100, 20
X_all = np.random.normal(size=(n, p))
beta = np.array([0.4**np.sqrt(j) for j in range(1, p+1)])
mu = X_all @ beta

n_reps = 200
mse_train = np.zeros((n_reps, p+1))
mse_test = np.zeros((n_reps, p+1))

for r in range(n_reps):
    eps = np.random.normal(size=n)
    eps_star = np.random.normal(size=n)
    y = mu + eps
    y_star = mu + eps_star

    for m in range(0, p+1):
        if m == 0:
            yhat_train_coef = y.mean()
            yhat = np.full(n, yhat_train_coef)
        else:
            Xm = X_all[:, :m]
            Xd = np.column_stack([np.ones(n), Xm])
            coef, *_ = np.linalg.lstsq(Xd, y, rcond=None)
            yhat = Xd @ coef

        mse_train[r, m] = np.mean((y - yhat)**2)
        mse_test[r, m] = np.mean((y_star - yhat)**2)

avg_train = mse_train.mean(axis=0)
avg_test = mse_test.mean(axis=0)

diffs = np.diff(mse_train, axis=1)
print(f"Train MSE violations: {(diffs > 1e-8).sum()} / {diffs.size} "
      f"(max diff = {diffs.max():.2e})")

ms = np.arange(0, p+1)
```

Train MSE violations: 0 / 4000 (max diff = -2.38e-09)

Setup: fixed 100×20 design, seed 43202, $\beta_j = 0.4^{\sqrt{j}}$, 200 reps, $m = 0, \dots, 20$ (combined plot with parts b/c below, since each part adds curves to the same figure).

Train MSE decreases monotonically ($1.364 \rightarrow 0.790$): nested models can only improve fit to the same training data. **Test MSE** dips to a minimum 1.109 at $m = 7$, then rises to 1.219 at $m = 20$ — classic overfitting, as predictors beyond the true signal add variance faster than they reduce bias. The train/test gap widens with m ($0 \rightarrow 0.43$).

Monotonicity check: 0/4000 within-run step comparisons violate train MSE decreasing (max “increase” 10^{-9} , floating-point noise), confirming the theoretical guarantee.

Part b

```
sigma2 = 1.0
B_m = np.zeros(p + 1)
for m in range(0, p + 1):
    Xm = np.ones((n, 1)) if m == 0 else np.column_stack([np.ones(n), X_all[:, :m]])
    coef_mu, *_ = np.linalg.lstsq(Xm, mu, rcond=None)
    B_m[m] = np.sum((mu - Xm @ coef_mu)**2)

m_vals = np.arange(0, p + 1)
theo_train = (B_m + (n - m_vals - 1) * sigma2) / n
theo_test = (B_m + (n + m_vals + 1) * sigma2) / n

diff_train = np.abs(avg_train - theo_train)
diff_test = np.abs(avg_test - theo_test)
print(f"Max |sim - theory| train: {diff_train.max():.4f} (at m={m_vals[diff_train.argmax()]})")
print(f"Max |sim - theory| test: {diff_test.max():.4f} (at m={m_vals[diff_test.argmax()]})")
```

```
Max |sim - theory| train: 0.0132 (at m=0)
Max |sim - theory| test: 0.0119 (at m=6)
```

Theory matches simulation closely (max diff 0.013 train, 0.012 test). B_m falls sharply for small m (signal is front-loaded by $\beta_j = 0.4\sqrt{j}$), while the variance term grows linearly with opposite sign for train ($-\sigma^2/n$) vs. test ($+\sigma^2/n$): bias dominates early (test MSE falls), variance dominates past $m \approx 7$ (test MSE rises); train MSE only sees the negative term, hence its monotonic decrease. The theoretical gap $2(m+1)\sigma^2/n$ grows linearly (0.02 $\rightarrow$ 0.42), exactly matching the simulated widening gap.

Part c

```
single_run_test = mse_test[0, :]

plt.figure(figsize=(5.8, 3.6))
plt.plot(ms, avg_train, marker='o', color='C0', label='Average Train MSE (sim)')
plt.plot(ms, theo_train, linestyle='--', color='C0', label='Theoretical E[Train MSE]')
plt.plot(ms, avg_test, marker='s', color='C1', label='Average Test MSE (sim)')
plt.plot(ms, theo_test, linestyle='--', color='C1', label='Theoretical E[Test MSE]')
plt.plot(ms, single_run_test, marker='^', color='C2', linestyle=':',
         label='Test MSE, single run (run 0)')
plt.xlabel('m (number of predictors)')
plt.ylabel('MSE')
plt.title('Train/Test MSE: Simulated, Theoretical, and Single-Run')
plt.legend(fontsize=7)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

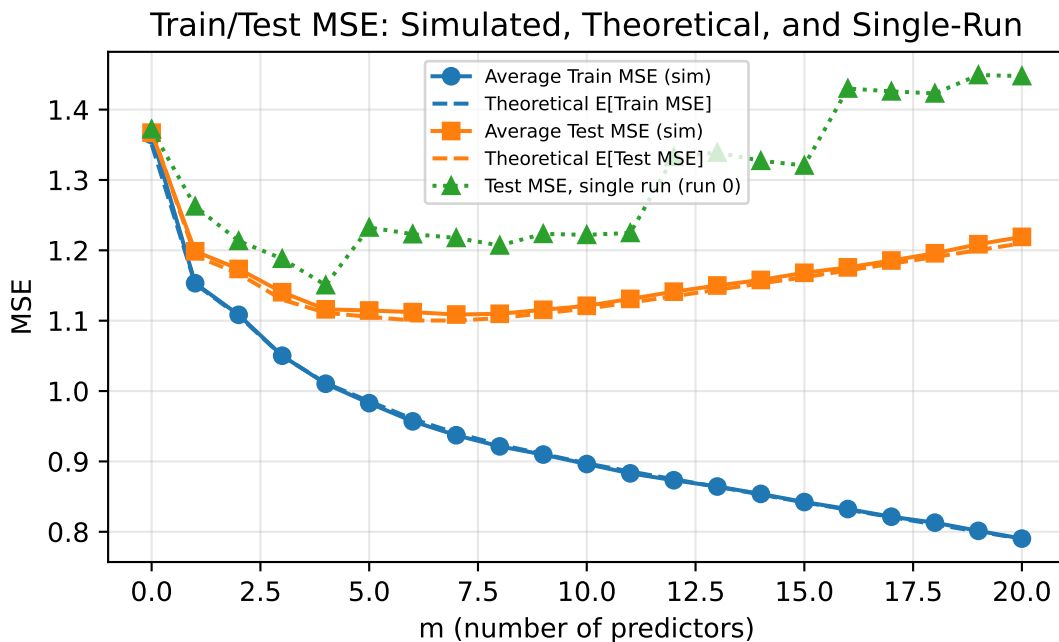


Figure 1: Combined plot for parts (a)-(c): simulated train/test MSE, theoretical $E[\text{MSE}]$, and a single-run test MSE curve

The single-run curve (run 0) is much bumpier than the average, with its own minimum near $m = 4$ and local jumps (near $m = 5, 12, 16$) instead of a clean U-shape. This is because $\text{MSE}_{\text{test},m}$ for one run is a single noisy draw (depends on the specific ϵ, ϵ^*), while averaging over 200 runs cancels that noise and converges to the smooth deterministic $E[\text{MSE}_{\text{test},m}]$.

This is why training MSE alone can't select m : $E[\text{MSE}_{\text{train},m}] = [B_m + (n - m - 1)\sigma^2]/n$ has no term that increases with m (both B_m and $(n - m - 1)\sigma^2$ shrink), so minimizing it always picks the largest m . $E[\text{MSE}_{\text{test},m}] = [B_m + (n + m + 1)\sigma^2]/n$ does include a rising variance term, which produces the U-shape — a signal training MSE never sees.

Question 2: Prediction error at one target point

Answer

```
n, p = 100, 6
i = np.arange(1, n + 1)[: , None]
j = np.arange(1, p + 1)[None, :]
X_all = np.sqrt(2) * np.cos(np.pi * j * (i - 0.5) / n)

beta = np.array([0.5]*p)
x0 = np.array([0, 1, 0, 0, 1, 0])
sigma2 = 1.0
mu0 = x0 @ beta
```

Part a

$\mu_0 = \mathbf{x}_0^\top \beta = 0.5 + 0.5 = 1.0$ (only coordinates 2 and 5 of $\mathbf{x}_0$ are nonzero).

m	Bias ($\sum_{j=m+1}^6 x_{0j}\beta_j$)	Squared bias	Variance term	Expected sq. error
0	1.0	1.00	0.01	1.01
1	1.0	1.00	0.01	1.01
2	0.5	0.25	0.02	0.27
3	0.5	0.25	0.02	0.27
4	0.5	0.25	0.02	0.27
5	0.0	0.00	0.03	0.03
6	0.0	0.00	0.03	0.03

The squared-bias term counts signal from *omitted* predictors ($j > m$) that $\mathbf{x}_0$ still loads on — it doesn't shrink with n . The variance term counts estimation noise from *included* predictors, growing with m since more coefficients are estimated (each weighted by x_{0j}^2).

Since $x_{0j} = 0$ for $j = 1, 3, 4, 6$, the error is unchanged at $m = 0 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 4, 5 \rightarrow 6$. It changes only when predictor 2 enters ($m = 1 \rightarrow 2$: bias $1.0 \rightarrow 0.25$) or predictor 5 enters ($m = 4 \rightarrow 5$: bias $0.25 \rightarrow 0$) — the only two coordinates $\mathbf{x}_0$ actually depends on.

Part b

```
m_vals = np.arange(0, p+1)
theo_bias2 = np.array([(np.sum(x0[m:]*beta[m:]))**2 for m in m_vals])
theo_var = np.array([(sigma2/n)*(1+np.sum(x0[m:])**2) for m in m_vals])
theo_error = theo_bias2 + theo_var

np.random.seed(43203)
n_reps = 200
sq_err = np.zeros((n_reps, p+1))

for r in range(n_reps):
    eps = np.random.normal(size=n)
    y = X_all @ beta + eps

    for m in range(0, p+1):
        if m == 0:
            Xm = np.ones((n, 1))
            x0m = np.array([1.0])
        else:
            Xm = np.column_stack([np.ones(n), X_all[:, :m]])
            x0m = np.concatenate([[1.0], x0[:m]])

        coef, *_ = np.linalg.lstsq(Xm, y, rcond=None)
        mu0_hat = x0m @ coef
        sq_err[r, m] = (mu0_hat - mu0)**2
```

```

avg_sq_err = sq_err.mean(axis=0)

diff = np.abs(avg_sq_err - theo_error)
print(f"Max |sim - theory|: {diff.max():.4f} (at m={m_vals[diff.argmax()]})")

best_m = m_vals[np.argmin(theo_error)]
print("Smallest m minimizing theoretical error:", best_m,
      "(value =", round(theo_error.min(), 4), ")")

plt.figure(figsize=(5.5, 3.2))
plt.plot(m_vals, avg_sq_err, marker='o', label=r'Simulated average  $(\hat{\mu}_{0,m} - \mu_0)^2$ ')
plt.plot(m_vals, theo_error, marker='s', linestyle='--', label='Theoretical expected squared error')
plt.xlabel('m (number of predictors)')
plt.ylabel('Squared error at  $x_0$ ')
plt.title('Simulated vs. Theoretical Prediction Error at  $x_0$ ')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Max |sim - theory|: 0.0155 (at m=0)
 Smallest m minimizing theoretical error: 5 (value = 0.03)

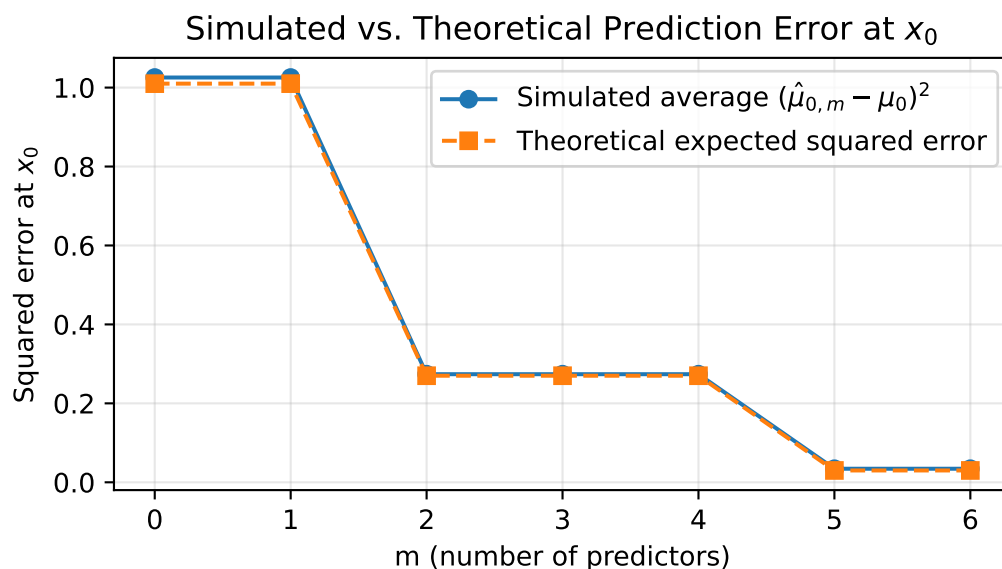


Figure 2: Simulated vs. theoretical prediction error at x_0

Simulated averages (200 reps, seed 43203) match theory closely: 1.03 vs. 1.01 ($m = 0, 1$), 0.27 vs. 0.27 ($m = 2, 3, 4$), 0.03 vs. 0.03 ($m = 5, 6$) — all diffs under 0.02, same step pattern as part (a) (flat except when predictor 2 or 5 enters). Theoretical error is minimized (tied) at $m = 5, 6$; smallest such m is 5.

Part c

Since $\mu_0 = \mathbf{x}_0^\top \beta = \sum_j x_{0j} \beta_j$ and $\mathbf{x}_0 = (0, 1, 0, 0, 1, 0)^\top$, only coordinates 2 and 5 are nonzero. So $\beta_1, \beta_3, \beta_4, \beta_6$ get multiplied by zero and don't affect μ_0 , even though they're genuinely nonzero in the true model. The same logic carries into the bias/variance formula: omitting predictor j only adds bias if $x_{0j} \beta_j \neq 0$, and including it only adds variance if $x_{0j}^2 \neq 0$. This is a property of this one target point, not of the true model itself.

Contrast with Question 1. There, test MSE averages error over an entire test set with iid $N(0, 1)$ covariates, so essentially every coordinate is nonzero somewhere in the average, and all 20 true predictors contribute. Question 2's error is local to one point and depends only on which coefficients that point actually “activates”; Question 1's is global and aggregates across the full covariate distribution.

Question 3: The optimism correction

Answer

Part a

Write $\mathbf{y} = \mu + \epsilon$, $\mathbf{y}^* = \mu + \epsilon^*$, independent, mean zero, covariance $\sigma^2 \mathbf{I}_n$ given $\mathbf{X}$. Since $\mathbf{H}$ is symmetric and idempotent, so is $\mathbf{I}_n - \mathbf{H}$.

Training MSE.

$$\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}$$

Squaring, using $(\mathbf{I}_n - \mathbf{H})^\top(\mathbf{I}_n - \mathbf{H}) = (\mathbf{I}_n - \mathbf{H})^2 = \mathbf{I}_n - \mathbf{H}$:

$$\|\mathbf{y} - \mathbf{H}\mathbf{y}\|_2^2 = \underbrace{\|(\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu}\|_2^2}_{B^2} + 2\boldsymbol{\mu}^\top(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon} + \boldsymbol{\epsilon}^\top(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}$$

Taking $E(\cdot \mid \mathbf{X})$: the cross term vanishes ($E(\boldsymbol{\epsilon} \mid \mathbf{X}) = \mathbf{0}$), and $E(\boldsymbol{\epsilon}^\top(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon} \mid \mathbf{X}) = \sigma^2 \text{tr}(\mathbf{I}_n - \mathbf{H}) = \sigma^2(n - (p + 1))$. So

$$E(\|\mathbf{y} - \mathbf{H}\mathbf{y}\|_2^2 \mid \mathbf{X}) = B^2 + \sigma^2(n - (p + 1)) \Rightarrow E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p + 1}{n}\right)$$

Test MSE.

$$\mathbf{y}^* - \mathbf{H}\mathbf{y} = \underbrace{(\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu}}_{\text{fixed}} + \underbrace{(\boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon})}_{\text{random}}$$

Squaring, the fixed/random cross term has conditional expectation zero (both $\boldsymbol{\epsilon}, \boldsymbol{\epsilon}^*$ are mean zero given $\mathbf{X}$), leaving $E(\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|_2^2 \mid \mathbf{X}) = B^2 + E(\|\boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}\|_2^2 \mid \mathbf{X})$. Expanding, using $\mathbf{H}^\top\mathbf{H} = \mathbf{H}^2 = \mathbf{H}$:

$$\|\boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}\|_2^2 = \boldsymbol{\epsilon}^{*\top}\boldsymbol{\epsilon}^* - 2\boldsymbol{\epsilon}^{*\top}\mathbf{H}\boldsymbol{\epsilon} + \boldsymbol{\epsilon}^\top\mathbf{H}\boldsymbol{\epsilon}$$

Term by term: $E(\boldsymbol{\epsilon}^{*\top}\boldsymbol{\epsilon}^* \mid \mathbf{X}) = \sigma^2 n$; the cross term is 0 since $\boldsymbol{\epsilon}, \boldsymbol{\epsilon}^*$ are independent and mean zero; $E(\boldsymbol{\epsilon}^\top\mathbf{H}\boldsymbol{\epsilon} \mid \mathbf{X}) = \sigma^2 \text{tr}(\mathbf{H}) = \sigma^2(p + 1)$. So

$$E(\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|_2^2 \mid \mathbf{X}) = B^2 + \sigma^2(n + p + 1) \Rightarrow E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p + 1}{n}\right)$$

The only difference between the two results is the sign on $\sigma^2(p + 1)/n$: training error “reuses” $\boldsymbol{\epsilon}$ to both fit and evaluate the model (subtracting a variance term), while test error pays an extra penalty since $\mathbf{H}\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon}^*$ are independent (adding one instead).

Part b

Subtracting the two results from part (a), the B^2/n term cancels entirely:

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{2\sigma^2(p + 1)}{n}$$

so the expected optimism depends only on the noise variance and $p + 1$ relative to n , not on how well-specified the model is.

Numerical values ($n = 120$, $p = 4$, $B^2/n = 0.04$, $\sigma^2 = 1$):

$$E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = 0.04 + \left(1 - \frac{5}{120}\right) = 0.9983, \quad E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = 0.04 + \left(1 + \frac{5}{120}\right) = 1.0817$$

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{2(5)}{120} = 0.0833$$

Why the ordering need not hold in a single realization. These are *expectations* over the distributions of $\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon}^*$, not guarantees about any one draw. For a specific realized pair, it's entirely possible for $\boldsymbol{\epsilon}^*$ to happen to align well with the fitted values, giving $\text{MSE}_{\text{test}} < \text{MSE}_{\text{train}}$ for that instance — even though, on average across repeated draws, test error is higher by construction (the model is evaluated on data it never saw). The inequality is a statement about long-run averages, not a pathwise guarantee, much like a fair coin being expected to land heads half the time without every flip doing so.

Part c

With $n = 120$, $p = 4$ ($p + 1 = 5$), $\text{RSS} = 116.4$, $\hat{\sigma}^2 = 0.96$:

$$\widehat{\text{MSE}}_{\text{test}} = \frac{116.4 + 2(5)(0.96)}{120} = \frac{126}{120} = 1.05, \quad C_p = \frac{116.4}{0.96} - 120 + 10 = 121.25 - 120 + 10 = 11.25$$

Relationship. From the definition of $\widehat{\text{MSE}}_{\text{test}}$: $n \cdot \widehat{\text{MSE}}_{\text{test}} = \text{RSS} + 2(p+1)\hat{\sigma}^2$. Dividing by $\hat{\sigma}^2$ and substituting $\text{RSS}/\hat{\sigma}^2 = C_p + n - 2(p+1)$:

$$\frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} = \frac{\text{RSS}}{\hat{\sigma}^2} + 2(p+1) = C_p + n \implies C_p = \frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} - n$$

Check: $\frac{120}{0.96}(1.05) - 120 = 131.25 - 120 = 11.25$.

Why minimizing C_p selects the same model. C_p is an affine function of $\widehat{\text{MSE}}_{\text{test}}$ with slope $n/\hat{\sigma}^2$ and intercept $-n$. When n and $\hat{\sigma}^2$ are shared across candidate models, that slope is a fixed positive constant, so the map is strictly increasing — it preserves ordering across models. Whichever model minimizes $\widehat{\text{MSE}}_{\text{test}}$ therefore also minimizes C_p : the two criteria are just different (monotonically related) scales for the same quantity.

Question 4: Comparing Mallows' C_p , AIC, and BIC

Answer

Part a

```
import pandas as pd

df = pd.read_csv('data/diabetes.csv')
train = df.iloc[0:370].reset_index(drop=True)
n = 370

def fit_ols(train, predictors, response='y'):
    X = train[predictors].values
    Xd = np.column_stack([np.ones(len(train)), X])
    y = train[response].values
    coef, *_ = np.linalg.lstsq(Xd, y, rcond=None)
    rss = np.sum((y - Xd @ coef)**2)
    return coef, rss

modelA_preds = ['bmi', 'bp', 's5', 'sex', 's1', 's2', 's4']
modelB_preds = modelA_preds + ['s6']
full_preds = ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']

coefA, rssA = fit_ols(train, modelA_preds)
coefB, rssB = fit_ols(train, modelB_preds)
coefF, rssF = fit_ols(train, full_preds)

pA, pB, pF = 7, 8, 10
df_full = n - (pF + 1)
sigma2_hat = rssF / df_full

print(f"Model A: p={pA}, RSS={rssA:.4f}")
print(f"Model B: p={pB}, RSS={rssB:.4f}")
print(f"Full : p={pF}, RSS={rssF:.4f}")
print(f"Full model residual df = {n}-{pF+1} = {df_full}")
print(f"sigma2_hat = {rssF:.4f}/{df_full} = {sigma2_hat:.4f}")
```

```
Model A: p=7, RSS=1098908.5656
Model B: p=8, RSS=1089717.0568
Full : p=10, RSS=1089451.7212
Full model residual df = 370-11 = 359
sigma2_hat = 1089451.7212/359 = 3034.6845
```

Model	p	RSS
A	7	1,098,908.57
B	8	1,089,717.06
Full	10	1,089,451.72

Full model residual df = $370 - 11 = 359$; $\hat{\sigma}^2 = 1,089,451.72/359 = 3034.6845$.

Part b

```
def cp(rss, p, n, sigma2_hat):
    return rss/sigma2_hat - n + 2*(p+1)

def aic_red(rss, p, n):
    return n*np.log(rss/n) + 2*(p+1)

def bic_red(rss, p, n):
    return n*np.log(rss/n) + (p+1)*np.log(n)

for name, rss, p in [("A", rssA, pA), ("B", rssB, pB)]:
    print(f"Model {name}: Cp={cp(rss,p,n,sigma2_hat):.4f}, "
          f"AIC={aic_red(rss,p,n):.4f}, BIC={bic_red(rss,p,n):.4f}")
```

Model A: Cp=8.1163, AIC=2974.6403, BIC=3005.9483

Model B: Cp=7.0874, AIC=2973.5325, BIC=3008.7540

Model	C_p	AIC ^{red}	BIC ^{red}
A	8.1163	2974.6403	3005.9483
B	7.0874	2973.5325	3008.7540

Model selected (smaller is better, within each criterion only): $C_p \rightarrow \mathbf{B}$ ($7.09 < 8.12$); AIC^{red} $\rightarrow \mathbf{B}$ ($2973.53 < 2974.64$); BIC^{red} $\rightarrow \mathbf{A}$ ($3005.95 < 3008.75$).

C_p and AIC both favor the larger Model B — adding **s6** reduces RSS enough to outweigh the extra parameter penalty under these two criteria — while BIC favors the smaller Model A, since its penalty term $(p+1)\log(n)$ grows faster with n and penalizes the additional predictor more heavily.

Part c

Fit improvement vs. penalty. Adding **s6** drops RSS by 9,191.51. In C_p units this is $\Delta\text{RSS}/\hat{\sigma}^2 \approx 3.03$, versus a penalty of 2 per parameter, so C_p favors Model B. AIC's fit-improvement term is $n\log(\text{RSS}_A/\text{RSS}_B) \approx 3.11$, again above its penalty of 2, so AIC also favors Model B. BIC compares that same ~ 3.11 improvement against a much larger penalty of $\log(370) \approx 5.91$, so BIC favors the smaller Model A instead.

Why they disagree. C_p and AIC use a fixed penalty (~ 2) regardless of n , while BIC's penalty grows with $\log(n)$ and exceeds 2 once n is even moderately large. **s6** clears the smaller bar but not BIC's larger one — BIC's general tendency to favor sparser models as n grows.

Why selection isn't truth. Each criterion optimizes a different target under different assumptions: C_p /AIC approximate predictive accuracy, while BIC approximates posterior model probability and is only asymptotically consistent for the true model if that model is actually among the candidates. All three models here are almost certainly simplifications of the real relationship, and RSS itself is computed from one sample and would vary with a different one. "Selected by criterion X" just means "best by that criterion's own objective," not verified as the true data-generating process.

Question 5: Validation and final test data

Answer

Part a

The misuse is using the final test data to select among the eleven models (comparing their eleven test MSEs and keeping the smallest), not the final step of reporting that number. The analyst's defense — that the final test data never touched coefficient estimation — is true but beside the point. The problem isn't that the test data influenced *fitting*; it's that it was used to *select among* eleven already-fitted models. Once test-set performance is used to make any decision, including which model to report, the test set has become part of the modeling procedure rather than held out from it. A held-out set only gives an honest error estimate for a model fully specified in advance, independent of that data — here, the choice of which of the eleven models to report *depends on* the test data, so it's no longer untouched, independent data for the final selected model.

Why the minimum of eleven test MSEs is optimistic. Each model's test MSE is a single noisy estimate of its true expected prediction error, subject to sampling variability from the 72 rows in the final test set ($72 = 442 - 371 + 1$) — the same kind of noise as the single-run test MSE curve in Question 1(c), or the optimism gap in Question 3. Even if no model were truly better than the others, each test MSE would still fluctuate randomly above and below its own true value. Picking the *minimum* across eleven such noisy estimates doesn't just find whichever model is genuinely best — it preferentially selects whichever model happened to get a favorable noise draw on this particular test set, a selection effect that grows worse the more models are compared.

Reporting that minimum as "the" estimate of future prediction error therefore overstates how well the selected model will actually perform on new data: the number reflects both genuine model quality and the test set's own random noise that the selection step has cherry-picked.

Part b

1. **Split the training data (rows 1–370) into five folds** of roughly equal size, chosen without reference to any model’s performance.
2. **For each candidate model $m = 0, 1, \dots, 10$:** for each of the five folds in turn, fit the m -predictor model on the other four folds and compute its MSE on the held-out fold. Average the five fold-level MSEs to get one cross-validated estimate, CV_m . This uses only the 370 training rows throughout — no fitting or evaluation ever touches the final 72 test rows.
3. **Select $\hat{m}$ minimizing CV_m over $m = 0, \dots, 10$.** Since this selection is based on repeated held-out estimates within the training data rather than a single evaluation on the final test set, the final test data plays no role in the choice.
4. **Refit the chosen model ($\hat{m}$ predictors) on *all* 370 rows of training data** (not just four of the five folds) to get the final coefficients — CV was only used to select m , not to produce the final fit.
5. **Only now touch the final test data:** evaluate this single, already-selected-and-refit model on rows 371–442, and report that MSE as the estimate of future prediction error.

The key distinction from the analyst’s original procedure: model selection happens entirely within the training data via cross-validation, so by the time the final test data is used, there is exactly one model under consideration rather than eleven, and no selection decision remains to bias the reported number optimistically. The test set is used exactly once, purely for evaluation of a model already locked in beforehand.

Part c

What can still be reported transparently. Once all eleven test MSEs have been examined, the final test set has effectively been “used up” — it can no longer serve as an honest estimate of future prediction error for whichever model looks best. But the eleven numbers aren’t worthless: the analyst can transparently report all eleven, alongside a clear description of the selection procedure followed, making explicit that the reported minimum was chosen *after* seeing all eleven results and is therefore expected to be optimistic (the same effect discussed in part (a)). Reporting the full set, not just the winner, at least lets a reader see how much variability there was across models and how much the “best” one actually stood out.

What a new, valid evaluation would require. A fresh set of data not used anywhere in this process — neither for fitting any of the eleven models nor for comparing their test MSEs — separate from both the training rows (1–370) and the already-examined test rows (371–442). With that untouched data, the analyst could evaluate the already-selected model exactly once, and only then would the resulting MSE be a fair, unbiased estimate of future prediction error.

Reference

James, Witten, Hastie, Tibshirani, and Taylor, [*An Introduction to Statistical Learning*](#), Chapters 3, 5, and 6.